

```

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WebServer.h>
#include <HTTPClient.h>
#include <Adafruit_Fingerprint.h>
#include "LittleFS.h"
#include "RTCLib.h"
#include "BluetoothSerial.h"
#include <Preferences.h>
#include <ArduinoJson.h>

// --- CONFIGURACIÓN DE PINES ---
#define RXD2 16
#define TXD2 17
#define LED_VERDE 12
#define LED_ROJO 13

// --- CONFIGURACIÓN FIREBASE ---
const char* FB_BASE_URL =
"https://asistencia-93328-default-rtdb.firebaseio.com";
const char* FB_ASISTENCIA =
"https://asistencia-93328-default-rtdb.firebaseio.com/asistencia.json";

// --- OBJETOS ---
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&Serial2);
RTC_DS3231 rtc;
BluetoothSerial SerialBT;
WebServer server(80);
Preferences prefs;

// --- VARIABLES GLOBALES ---
String ssid, pass;
unsigned long lastSync = 0;
unsigned long btTimer = 0;
unsigned long lastWifiRetry = 0;
bool btActivo = true;
bool btAutenticado = false;
bool ventanaAbierta = true;
bool wifiIniciado = false;

// --- PROTOTIPOS ---
void activarModoWifi();
void verificarConexion();
void procesarWifiBT(String msg);
void procesarFechaBT(String msg);
void realizarResetTotal();
void syncTempToFirebase();
void verificarHuellaAsistencia();
void handleGetUsers();
void handleEnrol();
int findFreeID();
bool captureStep(int step);
void blinkLED(int pin, int veces);

```

```

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\n[SISTEMA] --- INICIANDO DEBUG ---");

  // Bloqueo de auto-conexión fantasma
  WiFi.persistent(false);
  WiFi.disconnect(true);
  WiFi.mode(WIFI_OFF);
  delay(500);

  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);

  if (!LittleFS.begin(true)) Serial.println("[ERR] LittleFS");
  if (!rtc.begin()) Serial.println("[ERR] RTC");

  prefs.begin("wifi-config", true);
  ssid = prefs.getString("ssid", "Vilers-2");
  pass = prefs.getString("pass", "Pabada686");
  prefs.end();

  Serial2.begin(57600, SERIAL_8N1, RXD2, TXD2);
  finger.begin(57600);
  if (finger.verifyPassword()) Serial.println("[OK] Sensor huella listo");

  Serial.println("[BT] Iniciando Bluetooth...");
  SerialBT.begin("Laggersoft");
  btTimer = millis();
  // --- RUTAS PARA ARCHIVOS ESTÁTICOS ---
  // Estas líneas le dicen al ESP que cuando el HTML pida style.css o script.js,
  los busque en LittleFS
  server.serveStatic("/style.css", LittleFS, "/style.css");
  server.serveStatic("/script.js", LittleFS, "/script.js");

  server.on("/", []() {
    File file = LittleFS.open("/index.html", "r");
    if (!file) {
      Serial.println("[ERR] No se encontro index.html");
      server.send(404, "text/plain", "Falta index.html");
      return;
    }
    server.streamFile(file, "text/html");
    file.close();
  });
  server.on("/get_users", handleGetUsers);
  server.on("/enrol", handleEnrol);
}

void loop() {
  // 1. MANEJO BLUETOOTH
  if (btActivo) {
    if (ventanaAbierta || btAutenticado) {
      if (SerialBT.available()) {

```

```

String msg = SerialBT.readStringUntil('\n');
msg.trim();
Serial.printf("[BT DATA]: %s\n", msg.c_str());

if (msg == "OBIWANKENOBI") {
    btAutenticado = true;
    ventanaAbierta = false;
    SerialBT.println("Acceso OK.");
} else if (msg == "CORTARBT" && btAutenticado) {
    activarModoWifi();
} else if (btAutenticado) {
    if (msg.startsWith("SSIDPASS:")) procesarWifiBT(msg);
    if (msg.startsWith("DATETIME:")) procesarFechaBT(msg);
    if (msg == "RESET") realizarResetTotal();
}
}
}

if (ventanaAbierta && (millis() - btTimer > 20000)) {
    Serial.println("[BT] Timeout. Pasando a WiFi...");
    activarModoWifi();
}

// 2. MANEJO WIFI (Solo si el BT ya se cerró)
if (wifiIniciado) {
    verificarConexion(); // <--- Aquí estaba el choque
    server.handleClient();

    if (WiFi.status() == WL_CONNECTED && (millis() - lastSync > 30000)) {
        syncTempToFirebase();
        lastSync = millis();
    }
}

verificarHuellaAsistencia();
}

void activarModoWifi() {
    Serial.println("[RADIO] Cerrando BT e iniciando WiFi...");
    SerialBT.end();
    btActivo = false;

    WiFi.mode(WIFI_STA);
    delay(100);
    WiFi.begin(ssid.c_str(), pass.c_str());

    // IMPORTANTE: Seteamos lastWifiRetry al tiempo actual
    // para que verificarConexion() espere 20 segundos antes de reintentar.
    lastWifiRetry = millis() + 10000;

    server.begin();
    wifiIniciado = true;
}

```

```

void verificarConexion() {
    // Solo intenta reconectar si no está conectado Y ya pasaron 20 segundos desde
    el último intento
    if (WiFi.status() != WL_CONNECTED) {
        if (millis() - lastWifiRetry > 20000) {
            lastWifiRetry = millis();
            Serial.println("[WIFI] No conectado, reintentando WiFi.begin...");
            WiFi.begin(ssid.c_str(), pass.c_str());
        }
    } else {
        // Log de conexión exitosa una sola vez
        static bool logueado = false;
        if (!logueado) {
            Serial.print("[WIFI] Conectado exitosamente. IP: ");
            Serial.println(WiFi.localIP());
            logueado = true;
        }
    }
}

// --- RESTO DE FUNCIONES (Sin cambios, manteniendo logs) ---

void procesarWifiBT(String msg) {
    int c1 = msg.indexOf('['), c2 = msg.indexOf(']');
    int c3 = msg.lastIndexOf('['), c4 = msg.lastIndexOf(']');
    if (c1 != -1 && c3 != -1) {
        ssid = msg.substring(c1 + 1, c2);
        pass = msg.substring(c3 + 1, c4);
        prefs.begin("wifi-config", false);
        prefs.putString("ssid", ssid);
        prefs.putString("pass", pass);
        prefs.end();
        Serial.println("[OK] WiFi guardado. Reiniciando...");
        delay(500);
        ESP.restart();
    }
}

void procesarFechaBT(String msg) {
    int start = msg.indexOf('[') + 1;
    int end = msg.lastIndexOf(']');
    String valStr = msg.substring(start, end);
    int v[6];
    int count = 0;
    int lastPos = 0;
    for (int i = 0; i <= valStr.length(); i++) {
        if (i == valStr.length() || valStr[i] == ',') {
            v[count++] = valStr.substring(lastPos, i).toInt();
            lastPos = i + 1;
        }
        if (count == 6) break;
    }
    if (count == 6) {

```

```

        rtc.adjust(DateTime(v[0], v[1], v[2], v[3], v[4], v[5]));
        Serial.println("[OK] RTC Actualizado.");
    }
}

void realizarResetTotal() {
    finger.emptyDatabase();
    LittleFS.remove("/fichadas.csv");
    LittleFS.remove("/temp.csv");
    prefs.begin("wifi-config", false);
    prefs.clear();
    prefs.end();
    delay(500);
    ESP.restart();
}

void verificarHuellaAsistencia() {
    if (finger.getImage() == FINGERPRINT_OK) {
        if (finger.image2Tz() == FINGERPRINT_OK && finger.fingerFastSearch() ==
FINGERPRINT_OK) {
            DateTime now = rtc.now();
            String data = String(finger.fingerID) + "," + now.timestamp();
            Serial.printf("[HUELLA] ID:%d detectado\n", finger.fingerID);
            File f1 = LittleFS.open("/fichadas.csv", FILE_APPEND);
            f1.println(data);
            f1.close();
            File f2 = LittleFS.open("/temp.csv", FILE_APPEND);
            f2.println(data);
            f2.close();
            blinkLED(LED_VERDE, 1);
        } else {
            Serial.println("[HUELLA] No reconocida");
            blinkLED(LED_ROJO, 1);
        }
        delay(1000);
    }
}

void syncTempToFirebase() {
    if (!LittleFS.exists("/temp.csv")) return;

    File f = LittleFS.open("/temp.csv", "r");
    if (!f) return;

    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    bool todoOk = true;

    Serial.println("[SYNC] Iniciando sincronización...");

    while (f.available()) {
        String line = f.readStringUntil('\n');
        line.trim();
    }
}

```

```

    if (line.length() > 0) {
        http.begin(client, FB_ASISTENCIA);
        http.addHeader("Content-Type", "application/json");

        // Enviamos la línea como un objeto JSON
        String jsonPayload = "{\"fichada\": \"" + line + "\"}";
        int code = http.POST(jsonPayload);

        if (code == 200 || code == 201) {
            Serial.printf("[SYNC] Enviado: %s\n", line.c_str());
        } else {
            Serial.printf("[SYNC] Error en línea %s. Code: %d\n", line.c_str(),
code);
            todoOk = false; // Si falla una línea, marcamos que no fue perfecto
        }
        http.end();
    }
}
f.close();

// 3. Si todo se envió bien, borramos el archivo para empezar de cero
if (todoOk) {
    LittleFS.remove("/temp.csv");
    Serial.println("[SYNC] Todo enviado. Archivo temporal borrado.");
} else {
    Serial.println("[SYNC] Hubo errores. Los datos permanecen en el CSV para el
próximo intento.");
}
}

void handleGetUsers() {
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    http.begin(client, String(FB_BASE_URL) + "/tbl_alumnos.json");
    int code = http.GET();
    server.send(code, "application/json", http.getString());
    http.end();
}

void handleEnrol() {
    server.setHeader("Access-Control-Allow-Origin", "*"); // Para evitar error
de CORS
    String uid = server.arg("uid");
    int id = findFreeID();

    if (id == -1) {
        server.send(200, "text/plain", "No hay espacio para mas huellas");
        return;
    }

    // PASO 1: Captura inicial

```

```

if (!captureStep(1)) {
    server.send(200, "text/plain", "Error: No se detecto el dedo");
    return;
}
blinkLED(LED_VERDE, 1); // Feedback rápido de primera lectura
Serial.println("Paso 1 OK. Quite el dedo...");

// ESPERA: A que el usuario levante el dedo (Crucial para un buen enrolado)
unsigned long waitTime = millis();
while (finger.getImage() != FINGERPRINT_NOFINGER) {
    if (millis() - waitTime > 5000) break; // Timeout de seguridad
    delay(100);
}

Serial.println("Ponga el mismo dedo otra vez...");
delay(1000); // Pequeña pausa para que el sensor se limpie

// PASO 2: Confirmación
if (captureStep(2) && finger.createModel() == FINGERPRINT_OK &&
finger.storeModel(id) == FINGERPRINT_OK) {

    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    String url = String(FB_BASE_URL) + "/tbl_alumnos/" + uid + "/huellaId.json";

    http.begin(client, url);
    int httpStatusCode = http.PUT(String(id));

    if (httpStatusCode == 200) {
        server.send(200, "text/plain", "OK"); // Respuesta a la web
        blinkLED(LED_VERDE, 3);
        Serial.printf("Enrolado exitoso ID: %d\n", id);
    } else {
        server.send(200, "text/plain", "Error Firebase");
        blinkLED(LED_ROJO, 3);
    }
    http.end();

} else {
    server.send(200, "text/plain", "Fallo: Las huellas no coinciden");
    blinkLED(LED_ROJO, 2);
}
}

bool captureStep(int step) {
    int p = -1;
    unsigned long timeout = millis();
    while (p != FINGERPRINT_OK && (millis() - timeout < 2500)) { p =
finger.getImage(); }
    return (p == FINGERPRINT_OK && finger.image2Tz(step) == FINGERPRINT_OK);
}

```

```
int findFreeID() {
    for (int i = 1; i <= 1000; i++) {
        if (finger.loadModel(i) != FINGERPRINT_OK) return i;
    }
    return -1;
}

void blinkLED(int pin, int veces) {
    for (int i = 0; i < veces; i++) {
        digitalWrite(pin, HIGH);
        delay(200);
        digitalWrite(pin, LOW);
        delay(200);
    }
}
```